EXPERIMENT ASSESSMENT

ACADEMIC YEAR 2025-26

Course: Multidisciplinary Minor
Course code: MDC401
Year: SE SEM: IV

Experiment No. 8
AIM: Use XSLT (eXtensible Stylesheet Language Transformations) to convert XML data into an HTML table for better presentation.Create an XML file (books.xml) containing book details (Title, Author, Price). Write an XSL file (books.xsl) that transforms the XML data into an HTML table. Link the XSL with XML and open it in a browser to see the formatted output.
Name: Ashish Ramesh Patel
Roll Number: 55
Date of Performance:
Date of Submission:

Evaluation

Performance Indicator	Max. Marks	Marks Obtained
Performance	5	
Understanding	5	
Journal work and timely submission.	10	
Total	20	

Performance Indicator	Exceed Expectations (EE)	Meet Expectations (ME)	Below Expectations (BE)
Performance	5	3	2
Understanding	5	3	2
Journal work and timely submission.	10	8	4

Checked by

Name of Faculty: Dr. Yogesh Pingle

Aim: Use XSLT (eXtensible Stylesheet Language Transformations) to convert XML data into an HTML table for better presentation.Create an XML file (books.xml) containing book details (Title, Author, Price). Write an XSL file (books.xsl) that transforms the XML data into an HTML table. Link the XSL with XML and open it in a browser to see the formatted output.

Theory:

React is a popular JavaScript library used for building interactive user interfaces. Traditionally, React applications used **class components** to manage state and lifecycle methods. With the introduction of **React Hooks**, functional components can now manage state and side effects more efficiently.

1. **React Hooks**

Hooks are special functions in React that allow developers to "hook into" React features such as state and lifecycle methods from functional components. Hooks improve code readability, reusability, and reduce complexity.

2. **useState Hook**

The useState hook is used to manage **state** in functional components. State refers to data that can change over time and affects what is rendered on the screen.
In a counter application, the counter value is stored as state. Whenever the state changes, React automatically re-renders the component to reflect the updated value.

3. **useEffect Hook**

The useEffect hook is used to handle **side effects** in functional components. Side effects include:

- Updating the document title
- Fetching data
- Logging values
- Running code after component render

In this experiment, useEffect is used to update the browser title whenever the counter value changes, demonstrating how side effects are handled in React.

4. **Component Re-rendering**

Whenever the state changes using useState, React re-renders the component. The useEffect hook runs after rendering, ensuring that side effects are synchronized with the component's state.

Algorithm / Procedure

1. Start the experiment.
2. Create a new React application using create-react-app.
3. Open the project folder in VS Code.
4. Create a functional component for the Counter App.
5. Initialize counter state using useState.

6. Add buttons to increment and decrement the counter value.
7. Use useEffect to update the document title when the counter changes.
8. Display the counter value dynamically on the webpage.
9. Run the React application in the browser.
10. Stop the experiment.

Code:

▪ books.xml

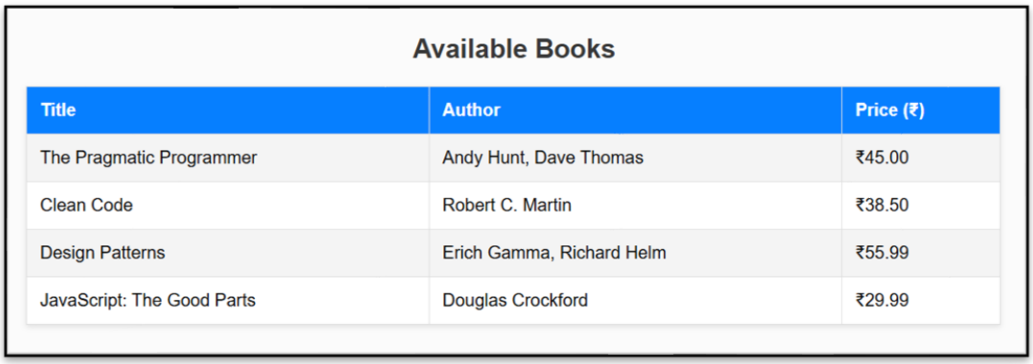
```
<?xml version="1.0" encoding="UTF-8"?>
<?xml-stylesheet type="text/xsl" href="books.xsl"?>
<catalog>
  <book>
    <title>The Pragmatic Programmer</title>
    <author>Andy Hunt, Dave Thomas</author>
    <price>45.00</price>
  </book>
  <book>
    <title>Clean Code</title>
    <author>Robert C. Martin</author>
    <price>38.50</price>
  </book>
  <book>
    <title>Design Patterns</title>
    <author>Erich Gamma, Richard Helm</author>
    <price>55.99</price>
  </book>
  <book>
    <title>JavaScript: The Good Parts</title>
    <author>Douglas Crockford</author>
    <price>29.99</price>
  </book>
</catalog>
```

▪ books.xsl

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:template match="/">
    <html>
    <head>
      <title>Book Catalog</title>
      <style>
        body { font-family: Arial, sans-serif; background-color: #f9f9f9; padding: 20px; }
        h2 { text-align: center; color: #333; }
```

```
            table { border-collapse: collapse; width: 60%; margin: 0 auto; background-color:
white; box-shadow: 0 2px 5px rgba(0,0,0,0.1); }
            th, td { border: 1px solid #ddd; padding: 12px; text-align: left; }
            th { background-color: #007bff; color: white; }
            tr:nth-child(even) { background-color: #f2f2f2; }
            tr:hover { background-color: #ddd; }
        </style>
    </head>
    <body>
        <h2>Available Books</h2>
        <table>
            <tr>
                <th>Title</th>
                <th>Author</th>
                <th>Price (₹)</th>
            </tr>

            <xsl:for-each select="catalog/book">
                <tr>
                    <td><xsl:value-of select="title"/></td>
                    <td><xsl:value-of select="author"/></td>
                    <td>₹<xsl:value-of select="price"/></td>
                </tr>
            </xsl:for-each>

        </table>
    </body>
    </html>
  </xsl:template>
</xsl:stylesheet>
```

Output:



Available Books

Title	Author	Price (₹)
The Pragmatic Programmer	Andy Hunt, Dave Thomas	₹45.00
Clean Code	Robert C. Martin	₹38.50
Design Patterns	Erich Gamma, Richard Helm	₹55.99
JavaScript: The Good Parts	Douglas Crockford	₹29.99

Conclusion

Thus, a simple counter application was successfully developed using React Hooks. The experiment demonstrates the use of useState for state management and useEffect for handling side effects. This approach simplifies component logic and improves code readability in modern React applications.

Questions:

1.What are React Hooks?

Ans:

Definition: React Hooks are special functions that let you use state and lifecycle features in functional components.

Key Points:

- Introduced in React 16.8.
- Common hooks: useState, useEffect, useContext.
- Simplify code by avoiding class components.
- Enable reusability of logic across components.

2.How does React re-render components?

Ans:

Definition: React re-renders components when their **state** or **props** change.

Key Points:

- Virtual DOM is updated and compared with previous DOM (diffing).
- Only changed parts are re-rendered (efficient updates).
- Hooks like useEffect run after re-render to handle side effects.
- Prevent unnecessary re-renders using React.memo or useCallback.